1) Concept Map : Algorithm ⇒ Growth function - Big O → Lower Bound → Best - Case Explain how Lower bound represent minimum execution time.

Algorithm → Growth Function → Big Omega → Lower Bound → Best case → performance guarantee

- **Algorithm**: A Sequence of steps to solve the problem
- **Growth Function**: Describes how the running time increases with input size
- **Big Omega**: Represents the lower bound of an algorithm
- **Lower Bound**: Indicates the minimum execution time for any input of size n
- **Best case**: The minimum no. of operation in most favourable case
- **performance guarantee**: Assures the Algorithm will never perform better than Ω.

How Lower Bound Represents Minimum Execution Time

Lower Bound (Ω) gives the least number of operations an algorithm must perform.

Ex: If $T(n) = \Omega(n)$, then at least n operations are Required even in the Best case.

Why Best - case Analysisis Useful

* Shows the fastest possible execution time
* Helps Evaluate the Algorithm under ideal condition
* Provides insight into maximum efficiency of the algorithm

2) Concept Map: Recurrence Relation → Substitution method → Expansion → Simplification → Final Complexity step by step transformation of Recurrence into closed form.

Recurrence Relation	Substitution Method	Expansion	Simplification	Guess	Verification	Final Complexity
$T(n)$			$(n+4) = Z$			$O(f(n))$

$T(n)$ → Substitution Method → Expansion → Simplification → Guess → Verification → Final Complexity

Step-by-step Transformation of Recurrence in Closed Form $T(n) = T(n-1)+1$

Step 1:

$T(n) = T(n-1)+1$

$= T(n-2)+1+1$

$= T(n-2)+2$

Step 2:

Expand Again

$T(n) = T(n-2)+2$

$= T(n-2)+2+1$

$= T(n-3)+3$

Step 3:

Continue

$T(n) = T(n-k)+k$

Step 4:

stop

$T(n) = T(1)+n-1$

If $T(1)=1$

$T(n) = 1+(n-1)$

$T(n) = n$

Step 5:

$T(n) = n$

$T(n) = O(n)$

Interpretation - Benefits of structured approach.

* The structured sequence (Guess → simplification → Verification)
* It helps to understand the pattern of Recursion
* It improves our ability to analyse recursive
* It enables us to determine the time Complexity

Guess → Expression → Simplification → Verification → Guess